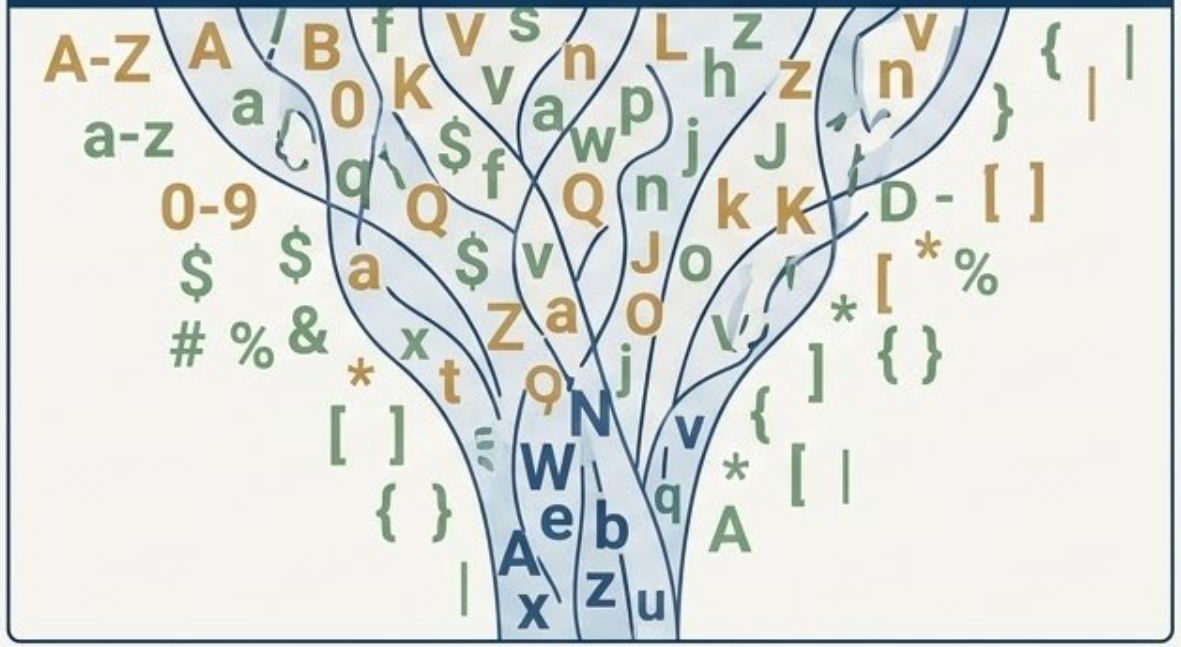
PowerShell Grund

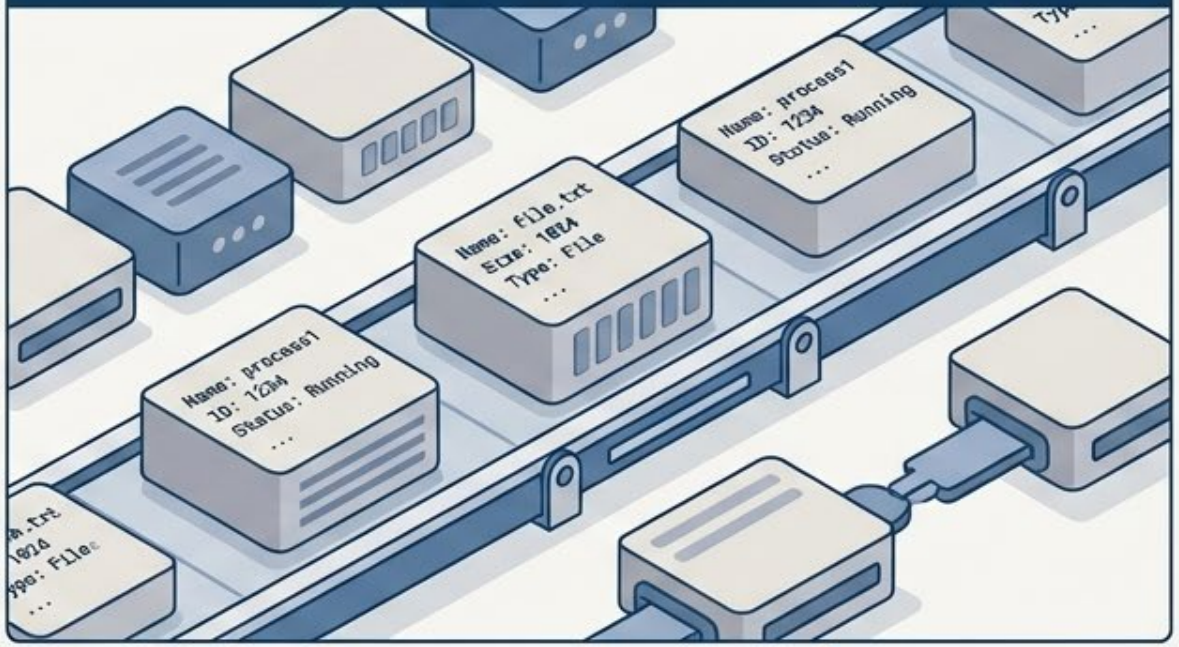
Arseniy Buzyan

Paradigmskiftet: Vad & Varför

Bash: Tolkar rå text



PowerShell: Hanterar objekt



PowerShell

1 Objektbaserat

Hanterar data med **egenskaper**, inte bara rå text.

2 Plattformsoberoende

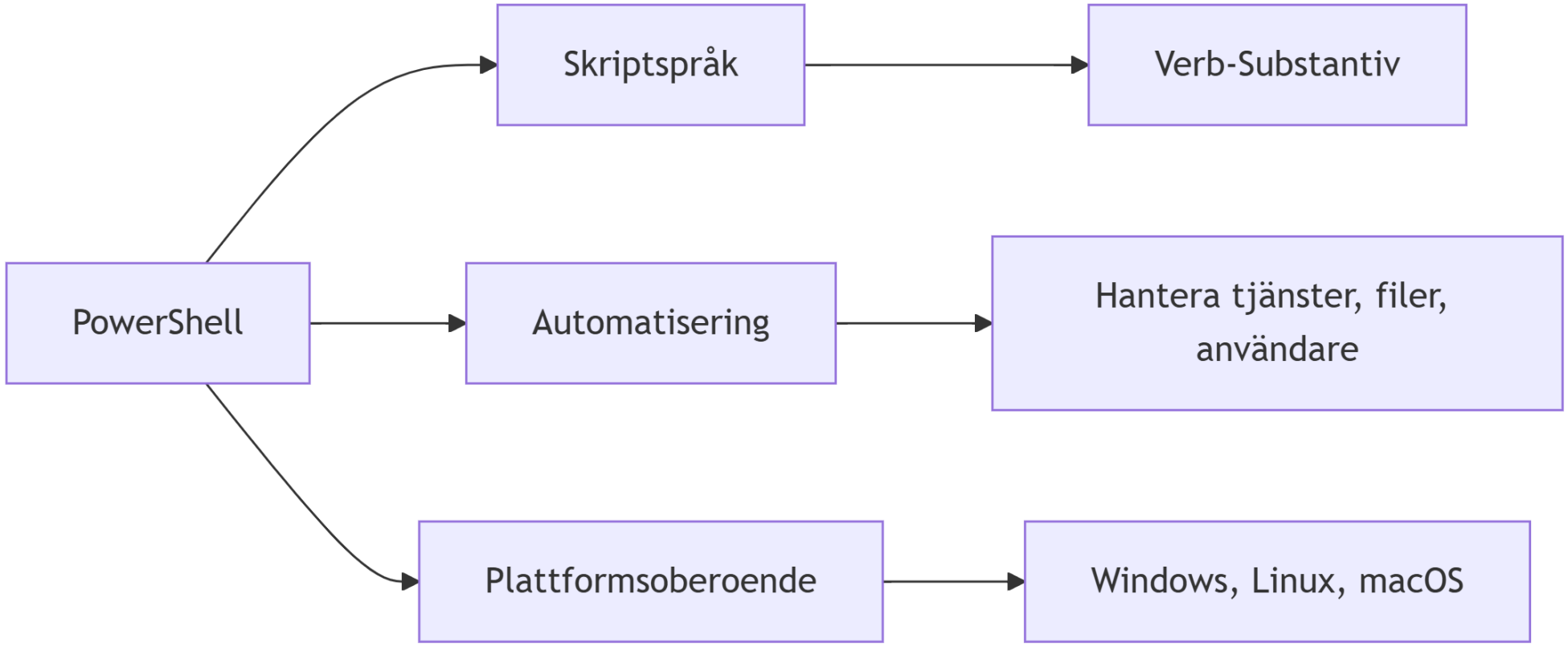
Windows, **Linux**, **macOS** (sedan 2016).

3 Standard

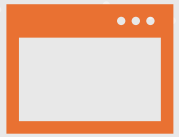
Inbyggt i **Windows** & **Azure**. Bygger på **.NET**.

Vad är PowerShell?

- PowerShell är ett modernt skriptspråk för automatisering – inte bara för Windows längre!
- Nyckelkoncept: PowerShell hanterar objekt, inte bara text. Det gör PowerShell kraftfullare än traditionella skriptspråk som Bash.



Varför PowerShell?



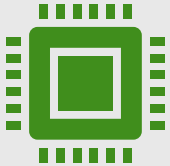
Standard i svenska Windows-miljöer



Kan hantera ALLT i Windows och Azure



Enkel att lära sig – Verb-Substantiv-logik



Öppen källkod och plattformsoberoende sedan 2016 – fungerar på Windows, Linux och macOS!



PowerShell 7.7 är den senaste versionen (2026) baserad på .NET 11.0, med förbättrad prestanda och nya funktioner för automatisering .

Cmdlets: Kommandonas Anatomi



Standardiserad logik (Verb-Substantiv). Inga obskyra förkortningar. Använd godkända verb (Get, Set, New) – aldrig synonymer som ”Delete”.

`Get-Service`	Hämta tjänster
`Start-Process`	Starta en process
`Stop-Service`	Stoppa en tjänst
`New-Item`	Skapa en ny fil/mapp
`Set-Content`	Skriv till en fil

Cmdlets har alltid formatet Verb-Substantiv:

- **Cmdlet** **Vad gör den?**
- Get-Service Hämta tjänster
- Start-Process Starta en process
- Stop-Service Stoppa en tjänst
- New-Item Skapa en ny fil/mapp
- Set-Content Skriv innehåll till en fil
- **Viktigt:** PowerShell har **godkända verb** som du bör använda för konsistens. Använd alltid `Get`, `Set`, `New`, `Remove` etc. – aldrig synonymer som `Delete` eller `Eliminate` .
- **Pipelining – Kedja ihop kommandon:**
- `Get-Service | Where-Object {$_.Status -eq "Running"}`
- Detta hämtar alla tjänster och filtrerar fram de som är igång.
- **Skillnad mot Bash:** PowerShell skickar **objekt** i pipelinen, inte text. Detta gör koden mer robust och läsbar. I Bash måste du använda `awk` och `sed` för att tolka text .

Variabler & Datatyper: Dataarkitektur



Pro-Tip: Variabler börjar alltid med $. Använd $variabel.GetType() för att inspektera en lådas innehåll.

Datatyp	Exempel	Förklaring
String	$namn = "Erik"	Text – alltid inom " "
Int	$ålder = 25	Heltal – inga citattecken
Boolean	$ärAdmin = $true	Sant eller falskt
Array	$servrar = @("WEB01","WEB02")	Lista med flera värden
Hashtable	$config = @{"Server"="WEB01"; "Port"=80}	Nyckel-värde-par

Exempel på datatyper

Tips: Använd $variabel.GetType() för att se vilken datatyp en variabel har.

Arrayer och Hashtables

Array = En lista med flera värden under samma namn.

```powershell
$servrar = @("WEB01", "WEB02", "SQL01")
$forsta = $servrar[0]
# WEB01 (index börjar på 0!)
```

Hashtable = Nyckel-värde-par (som en telefonbok).

```powershell
$config = @{
    "Server" = "WEB01"
    "Port" = 80
    "SSL" = $true
}
Write-Host $config["Server"]
# Skriver ut "WEB01"
```

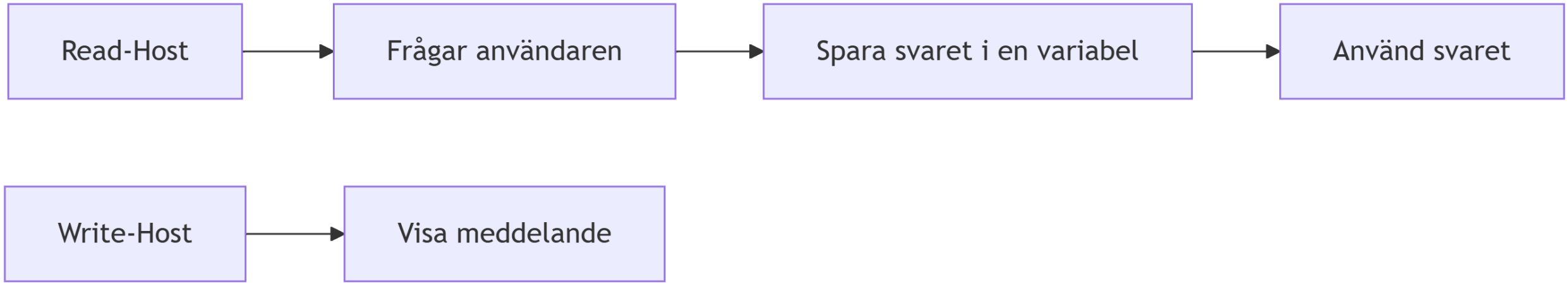
Jämförelse

Tänk: Array är som en hylla med numrerade lådor. Hashtable är som en telefonbok där du söker på namn . Hashtables är särskilt användbara som **uppslagstabeller** – du kan slå upp ett värde med en nyckel istället för att loopa igenom en lista .

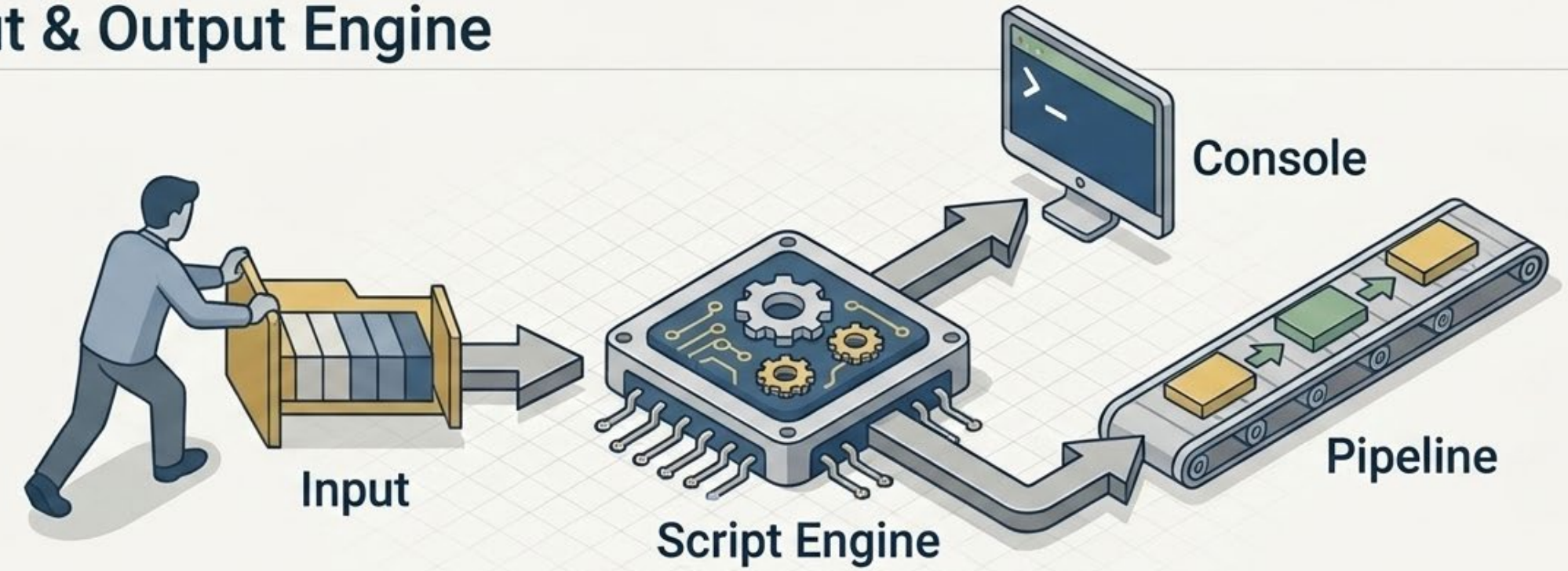
Array	**Hashtable**
@("a", "b", "c")	@{"Nyckel1"="Värde1"; "Nyckel2"="Värde2"}
Index: 0, 1, 2	Nyckel: "Server", "Port"
Används för listor	Används för konfiguration

Input och Output

- Input och Output
- Input: Fråga användaren med Read-Host
- Output: Visa meddelanden med Write-Host
- powershell
- `$namn = Read-Host "Vad heter du?"`
- `Write-Host "Hej $namn!"`



Input & Output Engine

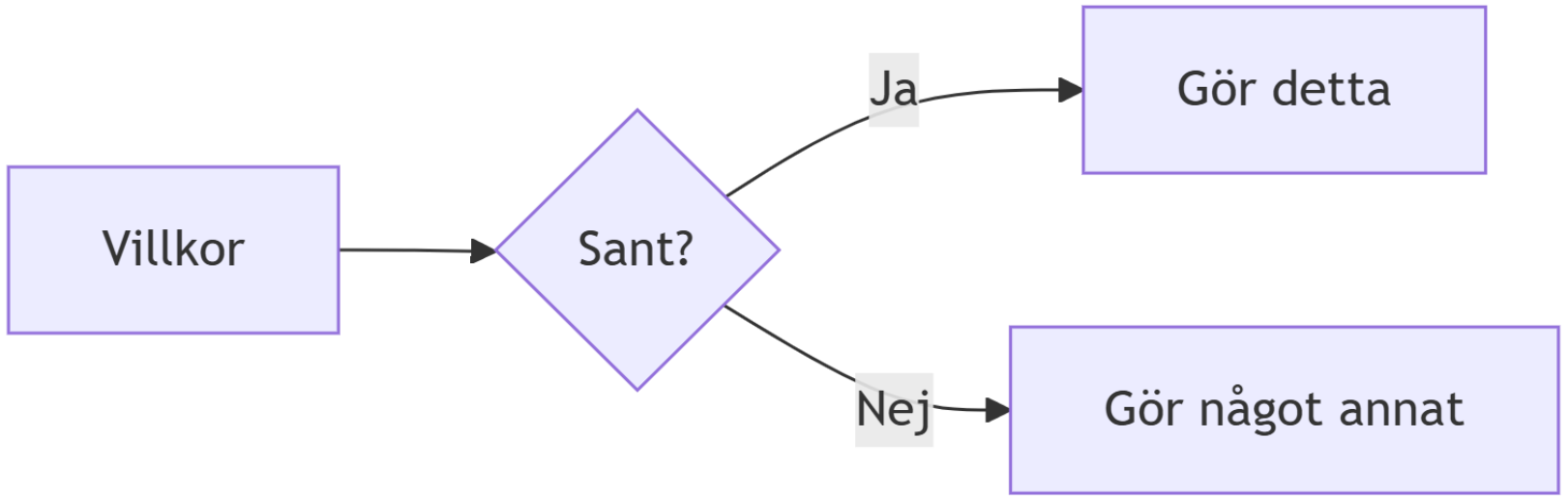


Read-Host	Input: Fråga användaren.
Write-Host	Output till konsol: Visar text för användaren, bryter pipelinen.
Write-Output	Output till pipeline: Skickar data vidare till nästa kommando.
Write-Warning	Varning: Skriver ut gul text.
Write-Error	Felmeddelande: Skriver ut röd text.

If / Else – Fatta Beslut

- Används för att styra flödet i ditt skript.
- powershell

```powershell
$ålder = Read-Host "Hur gammal är du?"
if ($ålder -ge 18) {
    Write-Host "Du är myndig!"
} else {
    Write-Host "Du är omyndig!"
}
```

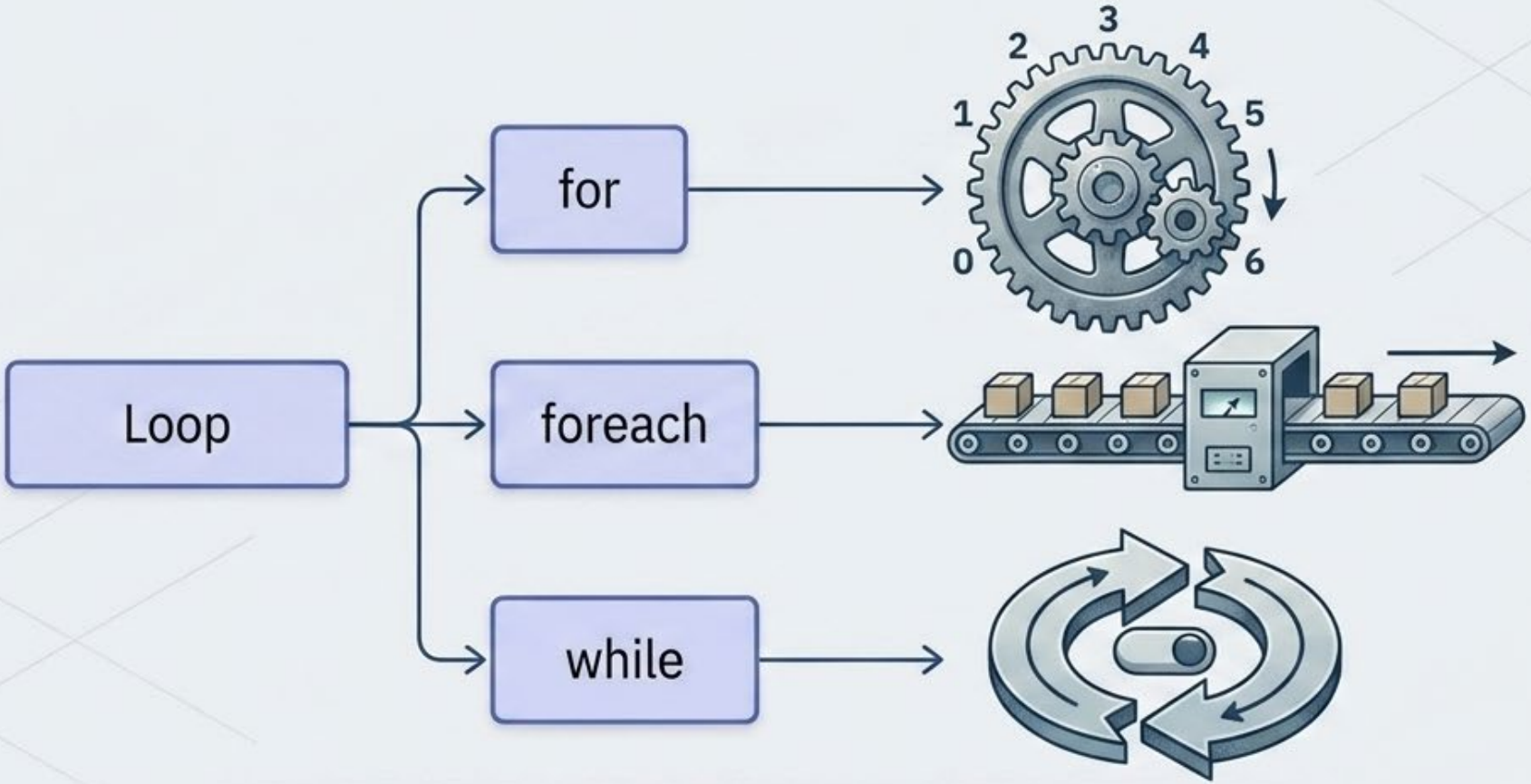


Jämförelseoperatorer

Tips: Placera alltid `$null` på vänster sida vid jämförelser: `if ($null -eq $variabel) { ... }` för att undvika oväntade fel .

Operator	Betydelse	Exempel
-eq	Lika med	`$age -eq 18`
-ne	Inte lika med	`$name -ne "Admin"`
-gt	Större än	`$age -gt 18`
-ge	Större eller lika med	`$age -ge 18`
-lt	Mindre än	`$age -lt 18`
-le	Mindre eller lika med	`$age -le 18`
-like	Mönstermatchning	`$name -like "WEB*"`

Loopar: Att Upprepa Kod




```
foreach ($key in $ageList.Keys) { ...
}
```

Loop

- For-loop – räkna till 5:

```
for ($i=1; $i -le 5; $i++) {
    Write-Host "Räknar: $i"
}
```

- Foreach-loop – gå igenom en lista:

```
$servrar = @("WEB01", "WEB02", "SQL01")
foreach ($server in $servrar) {
    Write-Host "Bearbetar $server"
}
```

- Foreach med Hashtable – iterera över nycklar:

```
$ageList = @{ "Kevin" = 36; "Alex" = 9 }
foreach ($key in $ageList.Keys) {
    Write-Host "$key är $($ageList[$key]) år gammal"
}
```

- Detta är ett vanligt mönster när du arbetar med hashtables som en samling .
- While-loop – kör tills villkor är sant:

```
$svar = ""
while ($svar -ne "ja") {
    $svar = Read-Host "Skriv 'ja' för att fortsätta"
}
```

- Tänk: Loopar är som att diska tallrikar – du gör samma sak om och om igen tills alla är klara.

Funktioner: Återanvändbara Recept




```powershell
function LäggTillMoms {
    param($Pris)
    return $Pris * 1.25
}

$prisMedMoms = LäggTillMoms -Pris 100
```

Återanvändbara

Skriv en gång, kör överallt (som ett recept).

Strukturerade

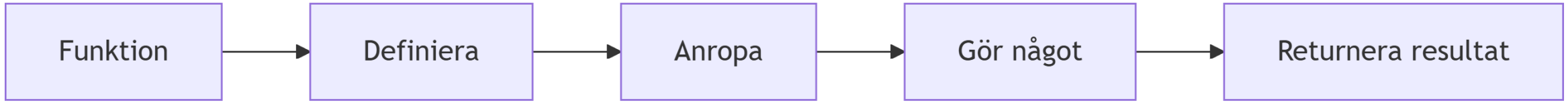
Gör koden mycket lättare att läsa.

Felsökning

Isolera logik i mindre, testbara delar.

Funktioner – Återanvändbar Kod

En funktion är ett kodblock som du kan använda om och om igen.



Exempel – en funktion som hälsar:

```powershell
function Hälsa {
    param($Namn)
    Write-Host "Hej $Namn!"
}
Hälsa -Namn "Erik"   # Skriver ut "Hej Erik!"
```

Exempel – en funktion med returvärde:

```powershell
function LäggTillMoms {
    param($Pris)
    return $Pris * 1.25
}
$prisMedMoms = LäggTillMoms -Pris 100
Write-Host "Pris med moms: $prisMedMoms"
```

Varför funktioner?

- Tänk: En funktion är som ett recept – du kan använda det om och om igen utan att skriva om det.

Fördel	**Förklaring**
Återanvändbar	Samma kod kan användas flera gånger
Strukturerad	Gör koden mer organiserad
Lättare att felsöka	Mindre delar är enklare att testa
Lättare att läsa	Tydligare vad koden gör

PowerShell vs Bash

- Båda är skriptspråk, men med olika styrkor

Egenskap	PowerShell	Bash
Plattform	Windows, Linux, macOS	Linux/Mac (och WSL2)
Datahantering	**Objekt-baserad**	Text-baserad
Pipelining	Skickar objekt	Skickar text (kräver awk/sed)
Syntax	Verb-Substantiv	Korta, symboliska kommandon
Felhantering	Try/Catch/Finally	Exit-koder och `set -e`
Läsbarhet	Mer lättläst	Mer kompakt
Integration	.NET, REST API, Windows	Unix-verktyg

Varför PowerShell är kraftfullare:



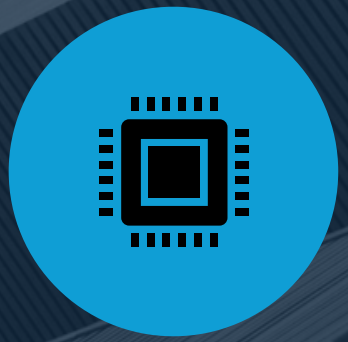
PowerShell-pipelinen
skickar **objekt** med egenskaper – du
behöver inte tolka text
med awk och sed som i Bash



PowerShell-skript är ofta **kortare och lättare att förstå** än motsvarande Bash-skript



PowerShell
är **plattformsoberoende** – samma
skript fungerar på Windows, Linux
och macOS



Tänk: PowerShell är som att prata med datorn med strukturerad data. Bash är som att tolka rå text. Båda har sin plats, men PowerShell är ofta kraftfullare för komplexa uppgifter .



Workshop – Skapa Ditt Första Skript

- **Mål:** Skapa ett enkelt PowerShell-skript som frågar användaren och fattar beslut.
- **Steg 1:** Skapa en fil `mittforstaskript.ps1`
- **Steg 2:** Skriv följande kod:
- powershell
- ```
  # Mitt första PowerShell-skript
  Write-Host "=== Välkommen! ==="
  $namn = Read-Host "Vad heter du?"
  $ålder = Read-Host "Hur gammal är du?"

  if ($ålder -ge 18) {
      Write-Host "Hej $namn! Du är myndig!"
  } else {
      Write-Host "Hej $namn! Du är omyndig!"
  }
  ```
- **Steg 3:** Spara och kör skriptet:
- powershell
- ```
  .\mittforstaskript.ps1
  ```
- **Steg 4:** Prova med olika åldrar (16, 18, 25) – vad händer?
- **Extra utmaning:** Lägg till en loop som frågar användaren om de vill fortsätta.
- **Tänk:** Detta är grunden till all PowerShell-automatisering – input, beslutsfattande och output.

Gör en quiz:

https://forms.gle/GhF9eKLG9J5t5Upu6